

Ankara Üniversitesi Bilgisayar Mühendisliği

BLM4522

AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ VİZE VE FİNAL PROJELERİ RAPORU

Ad Soyad: Emre Süt Öğrenci Numarası: 22290048

Vize Projeleri

Proje 5 - Veri Temizleme ve ETL Süreçleri Tasarımı

Proje 2 - Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Final Projeleri

Proje 1 - Veritabanı Performans Optimizasyonu ve İzleme

Proje 3 - Veritabanı Güvenliği ve Erişim Kontrolü

Proje 6 - Veritabanı Yükseltme ve Sürüm Yönetimi

GitHub Repo Linki: <https://github.com/emresut/Ag-Tabanli-Paralel-Dagitim-Sistemleri>

Bütün projeler tek reponun içinde bulunmaktadır. (5 projenin hepsi)

Proje 5 Youtube video linki: <https://www.youtube.com/watch?v=ru98t8VcT-Y>

Proje 2 Youtube video linki: <https://www.youtube.com/watch?v=gOWnCZRAydg&t=1s>

Proje 1 Youtube video linki: <https://www.youtube.com/watch?v=QD2mwkCVdaw>

Proje 3 Youtube video linki: <https://www.youtube.com/watch?v=vvzgEJHLORw>

Proje 6 Youtube video linki: <https://www.youtube.com/watch?v=PGDPbX4FQJs>

İÇİNDEKİLER

PROJE 5 - Veri Temizleme ve ETL Süreçleri Tasarımı	1
1 - Giriş	1
2 - Veri Seti	1
3 - ETL Süreci (Extract, Transform, Load)	2
3.1. Veri Alma	2
3.2. Veriyi Bozma	2
3.3. Veriyi Temizleme ve Dönüştürme	3
3.4. Temiz Veriyi Aktarma	3
4 - Veri Kalitesi Analizi	4
5 - Sonuç ve Genel Değerlendirme	4
PROJE 2 - Veritabanı Yedekleme ve Felaketten Kurtarma Planı	5
1 - Giriş	5
2 - Yedekleme Türleri	5
3 - Genel Akış	6
3.1. Yedekleme İşlemleri	6
3.2. Yedek Doğrulama ve Yedek İçeriğinin İncelenmesi	6
3.3. Felaket Senaryosu Durumunda Veritabanının Geri Yüklenmesi	7
3.4. Kurtarma Sonrası Kontrol	7
4 - Sonuç ve Değerlendirmeler	7
PROJE 1 - Veritabanı Performans Optimizasyonu ve İzleme	8
1 - Giriş	8
2 - Veri Seti	8
3 - Performans Analizi ve Optimizasyon Süreci	9
3.1. Başlangıç Durumu	9
3.2. Dynamic Management Views (DMV) ile Performans Takibi	9
3.3. İndeks Yönetimi ve Sorgu İyileştirme	10
4 - Sonuç ve Genel Değerlendirme	10

PROJE 3 - Veritabanı Güvenliği ve Erişim Kontrolü	11
1 - Giriş	11
2 – Rol ve Erişim Yönetimi	11
2.1. Rol Oluşturma	11
2.2. Kullanıcı Görünümleri ile Hassas Verilerin Korunması	11
2.3. Rol Yetkilendirme	12
3 – SQL Injection	12
3.1. Güvensiz Dinamik SQL Sorguları	13
3.2. Parametrelili Sorgular ile Saldırıdan Korunma	13
4 - Audit Logları ve Kullanıcı İşlemlerinin Takibi	14
5 - Sonuç ve Genel Değerlendirme	14
PROJE 6 - Veritabanı Yükseltme ve Sürüm Yönetimi	15
1 - Giriş	15
2 - Veritabanı Sürüm Yönetimi ve Sürüm Geçiş Süreci	15
2.1. Sürüm v1	15
2.2. Sürüm v2	16
2.3. Sürüm v3	16
3 – Geri Dönüş (Rollback) Planı	16
4 - DDL Trigger ile Şema Değişikliklerinin Takibi	17
5 - Sonuç ve Genel Değerlendirme	17

PROJE 5 - Veri Temizleme ve ETL Süreçleri Tasarımı

1) Giriş

Bu projede, gerçek hayatta sıkça karşılaşılan kirli verilerin temizlenmesi ve işlenmesi amacıyla bir ETL (Extract, Transform, Load) süreci gerçekleştirilmiştir. Veri analizi ve veri bilimi çalışmalarında, kullanılan verinin kalitesi elde edilecek sonuçların doğruluğunu doğrudan etkilediğinden, ETL süreci kritik bir öneme sahiptir.

Proje kapsamında Kaggle platformundan elde edilen bir çalışan(employee) veri seti kullanılmıştır. Bu veri seti önce veritabanına aktarılmış, ardından bilinçli olarak hatalar eklenerek “messy” (bozulmuş) hale getirilmiş ve son aşamada çeşitli SQL işlemleri ile temizlenerek “clean” (temiz) bir tabloya dönüştürülmüştür.

Veri seti linki: <https://www.kaggle.com/datasets/desolution01/messy-employee-dataset?resource=download>

Tüm süreçte MSSQL kullanılmıştır, araç olarak SQL Server Management Studio kullanılmıştır.

2) Veri Seti

Projede kullanılan veri seti Kaggle platformundan elde edilen “Messy Employee Dataset” adlı veri setidir. Bu veri seti fazlaca çalışan kaydı içermektedir.

Veri setinde bulunan sütunlar şu şekildedir: *Employee_ID, First_Name, Last_Name, Age, Department_Region, Status, Join_Date, Salary, Email, Phone, Performance_Score, Remote_Work*

Bu veri seti başlangıçta temiz bir yapıda sayılabilir ancak proje kapsamında veri temizleme işlemlerini gösterebilmek için üzerinde kasıtlı olarak çeşitli bozulmalar gerçekleştirilecektir. Raw veri setinde sadece yaş(age) kısmında boşluklar vardır.

Aşağıda örnek olması amacıyla raw veri setinin ilk 10 satırı görsel olarak verilmiştir.

	Employee_ID	First_Name	Last_Name	Age	Department_Region	Status	Join_Date	Salary	Email	Phone	Performance_Score	Remote_Work
1	EMP1000	Bob	Davis	25	DevOps-California	Active	4/2/2021	59767.65	bob.davis@example.com	-1651623197	Average	TRUE
2	EMP1001	Bob	Brown	NULL	Finance-Texas	Active	7/10/2020	65304.66	bob.brown@example.com	-1898471390	Excellent	TRUE
3	EMP1002	Alice	Jones	NULL	Admin-Nevada	Pending	12/7/2023	88145.9	alice.jones@example.com	-5596363211	Good	TRUE
4	EMP1003	Eva	Davis	25	Admin-Nevada	Inactive	11/27/2021	69450.99	eva.davis@example.com	-3476490784	Good	TRUE
5	EMP1004	Frank	Williams	25	Cloud Tech-Florida	Active	1/5/2022	109324.61	frank.williams@example.com	-1586734256	Poor	FALSE
6	EMP1005	Alice	Garcia	40	Sales-Texas	Inactive	6/10/2020	88642.84	alice.garcia@example.com	-5409003485	Good	FALSE
7	EMP1006	Frank	Jones	NULL	Admin-Nevada	Active	4/3/2020	96288.43	frank.jones@example.com	-4518376063	Good	FALSE
8	EMP1007	Bob	Jones	30	Cloud Tech-Florida	Inactive	7/17/2022	94497.91	bob.jones@example.com	-4134327559	Average	TRUE
9	EMP1008	Frank	Davis	35	Admin-Nevada	Inactive	12/8/2023	115565.82	frank.davis@example.com	-4177656123	Excellent	TRUE
10	EMP1009	Charlie	Johnson	NULL	DevOps-New York	Active	8/4/2022	76561.88	charlie.johnson@example.com	-8156985699	Excellent	TRUE

3) ETL Süreci (Extract, Transform, Load)

3.1. Veri Alma

Veri seti CSV formatında Kaggle üzerinden indirilmiş ve server içerisine import edilerek raw_employee_table tablosuna aktarılmıştır. Bu işlem sırasında SQL Server'ın Import Wizard aracı kullanılmıştır. Sonrasında raw veri setinin bozulmaması için raw_employee_table aynen messy_employee_table adlı bir table'ın içerisine kopyalanmıştır. Veriyi bozma işlemi messy_employee_table üzerinden yapılacaktır.

3.2. Veriyi Bozma

Öncelikle 01_load_raw_to_messy.sql ile raw veri olduğu gibi messy_employee_table tablosuna kopyalanmış. Bu tabloda veri, temizleme sürecini simüle edebilmek için veri bilinçli olarak bozulmuştur. Bu 02_disrupt_data.sql scripti ile gerçekleştirilmiştir.

Bu script içerisinde bazı age alanlarına "unknown" değeri atanmıştır, bazı email alanları "invalid_email" olarak değiştirilmiştir, kimi first_name'lerde başına ve sonuna bilerek boşluk karakteri konulmuştur, bazı salary(maaş) kısımlarına olması imkansız negatif değer verilmiştir (-5000), kimi yerde phone alanı "no-phone" yapılmıştır, Department_Region formatı normal tire olması gerekirken bazı yerlerde "Admin_Nevada" gibi alt tirenin olduğu yanlış formattaki versiyonlar konulmuştur. Bazı yerlerde Join_Date alanına "not_a_date" denilmiştir.

Aşağıda örnek olması amacıyla messy_employee_table tablosunun ilk 30 satırı görsel olarak verilmiştir.

	Employee_ID	First_Name	Last_Name	Age	Department_Region	Status	Join_Date	Salary	Email	Phone	Performance_Score	Remote_Work
1	EMP1000	Bob	Davis	25	DevOps-California	Active	4/2/2021	59767.65	bob.davis@example.com	-1651623197	Average	TRUE
2	EMP1001	Bob	Brown	NULL	Admin_Nevada	Active	7/10/2020	65304.66	bob.brown@example.com	-1898471390	Excellent	TRUE
3	EMP1002	Alice	Jones	NULL	Admin-Nevada	Pending	12/7/2023	88145.9	alice.jones@example.com	-5596363211	Good	TRUE
4	EMP1003	Eva	Davis	25	Admin-Nevada	Inactive	not_a_date	69450.99	invalid_email	-3476490784	Good	TRUE
5	EMP1004	Frank	Williams	25	Cloud Tech-Florida	Active	1/5/2022	109324.61	frank.williams@example.com	-1586734256	Poor	FALSE
6	EMP1005	Alice	Garcia	unknown	Sales-Texas	Inactive	6/10/2020	88642.84	alice.garcia@example.com	-5409003485	Good	FALSE
7	EMP1006	Frank	Jones	NULL	Admin-Nevada	Active	4/3/2020	96288.43	frank.jones@example.com	-4518376063	Good	FALSE
8	EMP1007	Bob	Jones	30	Cloud Tech-Florida	Inactive	7/17/2022	-5000	bob.jones@example.com	no-phone	Average	TRUE
9	EMP1008	Frank	Davis	35	Admin-Nevada	Inactive	12/8/2023	115565.82	frank.davis@example.com	-4177656123	Excellent	TRUE
10	EMP1009	Charlie	Johnson	NULL	DevOps-New York	Active	8/4/2022	76561.88	charlie.johnson@example.com	-8156985699	Excellent	TRUE
11	EMP1010	David	Williams	35	Cloud Tech-New York	Active	7/6/2024	72081.71	david.williams@example.com	-2720634590	Average	FALSE
12	EMP1011	David	Davis	NULL	Admin_Nevada	Active	9/4/2023	84946.19	david.davis@example.com	-1238296576	Poor	TRUE
13	EMP1012	Heidi	Williams	30	HR-New York	Active	3/5/2024	89295.77	heidi.williams@example.com	-2429543164	Excellent	FALSE
14	EMP1013	Grace	Williams	NULL	Finance-California	Active	not_a_date	85666.84	invalid_email	-2669300892	Good	FALSE
15	EMP1014	Alice	Garcia	25	Admin-Nevada	Active	6/10/2024	97633.68	alice.garcia@example.com	-1734206968	Excellent	TRUE
16	EMP1015	Grace	Jones	unknown	Sales-Florida	Pending	5/29/2021	117975.49	grace.jones@example.com	-4215291085	Good	TRUE
17	EMP1016	Alice	Brown	25	DevOps-Illinois	Inactive	4/23/2024	94867.33	alice.brown@example.com	-7713334475	Poor	FALSE
18	EMP1017	Grace	Garcia	30	Finance-Texas	Pending	11/28/2024	-5000	grace.garcia@example.com	no-phone	Average	TRUE
19	EMP1018	Charlie	Garcia	NULL	Cloud Tech-Nevada	Inactive	9/19/2024	94726.46	charlie.garcia@example.com	-2044397358	Excellent	FALSE
20	EMP1019	Charlie	Williams	35	Admin-Illinois	Pending	1/2/2020	91926.12	charlie.williams@example.com	-7107256365	Average	TRUE
21	EMP1020	Bob	Miller	35	HR-New York	Active	5/8/2021	111460.67	bob.miller@example.com	-9411800536	Excellent	TRUE
22	EMP1021	Bob	Jones	40	Admin_Nevada	Active	9/19/2020	96182.56	bob.jones@example.com	-5990048963	Good	FALSE
23	EMP1022	Charlie	Johnson	35	Sales-Illinois	Inactive	3/9/2021	115023.94	charlie.johnson@example.com	-6528092427	Average	TRUE
24	EMP1023	David	Garcia	35	Finance-Nevada	Pending	not_a_date	80667.01	invalid_email	-7572158623	Excellent	FALSE
25	EMP1024	Bob	Brown	30	Admin-Texas	Inactive	2/13/2020	91181.61	bob.brown@example.com	-1502745643	Good	TRUE
26	EMP1025	David	Miller	unknown	Cloud Tech-Nevada	Pending	4/30/2020	66025.92	david.miller@example.com	-9485625727	Average	FALSE
27	EMP1026	Frank	Miller	25	Sales-New York	Inactive	10/2/2023	83979.01	frank.miller@example.com	-7032032073	Average	FALSE
28	EMP1027	Heidi	Johnson	30	Finance-Florida	Active	7/12/2020	-5000	heidi.johnson@example.com	no-phone	Excellent	TRUE
29	EMP1028	Grace	Garcia	40	Finance-Nevada	Active	10/11/2023	95741.88	grace.garcia@example.com	-7712151341	Excellent	FALSE
30	EMP1029	Alice	Brown	40	DevOps-California	Active	5/24/2021	63411.31	alice.brown@example.com	-2849939088	Excellent	TRUE

3.3. Veriyi Temizleme ve Dönüştürme

Bu aşama projenin en kritik kısmıdır, ETL'de en önemli noktalar buradadır. Burası 04_create_empty_clean_table scriptinden sonra gelen 05_main.sql dosyasında gerçekleştirilmiştir.

Bu script içerisinde, first_name'lerde baştaki ve sondaki boşluklar eğer varsa temizlenmiştir, geçersiz sayısal değerler NULL yapılmıştır, geçersiz tarih formatları kontrol edilmiştir, email'ler gerçekten email formatına uygun mu diye kontrol edilmiştir değilse NULL verilmiştir, Department_Region alanı artık tek bir sütun değil de Department ve Region olmak üzere iki sütun olacak şekilde ayrılmıştır(bu bir veri dönüştürme örneği), Remote_Work alanı TRUE FALSE şeklindeyken bit tipine(1 0) dönüştürülmüştür(burada da bir veri dönüştürme vardır).

Doğruluğu kesin olmayan verilerde tahmin etmek yerine NULL vermenin daha doğru bir yaklaşım olacağı öngörülmüştür. Sonradan içinde NULL olan satırlar silinedebilir, bu bir tercihtir.

Tüm bu işlemler sonucunda veri analize daha uygun hale getirilmiştir.

3.4. Temiz Veriyi Aktarma

Temizleme işlemlerinden sonra veriler clean_employee_table tablosuna aktarılmıştır. Bu yapı sayesinde orijinal veri korunmuş, temiz veri ayrı bir tabloda tutulmuştur. Verilerin doğru hedef tabloya yüklenmesi bölümü de bu şekilde tamamlanmıştır.

Aşağıda örnek olması amacıyla clean_employee_table tablosunun ilk 30 satırı görsel olarak verilmiştir.

	Employee_ID	First_Name	Last_Name	Age	Department	Region	Status	Join_Date	Salary	Email	Phone	Performance_Score	Remote_Work
1	EMP1000	Bob	Davis	25	DevOps	California	Active	2021-04-02	59767.65	bob.davis@example.com	-1651623197	Average	1
2	EMP1001	Bob	Brown	NULL	Admin	Nevada	Active	2020-07-10	65304.66	bob.brown@example.com	-1898471390	Excellent	1
3	EMP1002	Alice	Jones	NULL	Admin	Nevada	Pending	2023-12-07	88145.90	alice.jones@example.com	-5596363211	Good	1
4	EMP1003	Eva	Davis	25	Admin	Nevada	Inactive	NULL	69450.99	NULL	-3476490784	Good	1
5	EMP1004	Frank	Williams	25	Cloud Tech	Florida	Active	2022-01-05	109324.61	frank.williams@example.com	-1586734256	Poor	0
6	EMP1005	Alice	Garcia	NULL	Sales	Texas	Inactive	2020-06-10	88642.84	alice.garcia@example.com	-5409003485	Good	0
7	EMP1006	Frank	Jones	NULL	Admin	Nevada	Active	2020-04-03	96288.43	frank.jones@example.com	-4518376063	Good	0
8	EMP1007	Bob	Jones	30	Cloud Tech	Florida	Inactive	2022-07-17	NULL	bob.jones@example.com	NULL	Average	1
9	EMP1008	Frank	Davis	35	Admin	Nevada	Inactive	2023-12-08	115565.82	frank.davis@example.com	-4177656123	Excellent	1
10	EMP1009	Charlie	Johnson	NULL	DevOps	New York	Active	2022-08-04	76561.88	charlie.johnson@example.com	-8156985699	Excellent	1
11	EMP1010	David	Williams	35	Cloud Tech	New York	Active	2024-07-06	72081.71	david.williams@example.com	-2720634590	Average	0
12	EMP1011	David	Davis	NULL	Admin	Nevada	Active	2023-09-04	84946.19	david.davis@example.com	-1238296576	Poor	1
13	EMP1012	Heidi	Williams	30	HR	New York	Active	2024-03-05	89295.77	heidi.williams@example.com	-2429543164	Excellent	0
14	EMP1013	Grace	Williams	NULL	Finance	California	Active	NULL	85666.84	NULL	-2669300892	Good	0
15	EMP1014	Alice	Garcia	25	Admin	Nevada	Active	2024-06-10	97633.68	alice.garcia@example.com	-1734206968	Excellent	1
16	EMP1015	Grace	Jones	NULL	Sales	Florida	Pending	2021-05-29	117975.49	grace.jones@example.com	-4215291085	Good	1
17	EMP1016	Alice	Brown	25	DevOps	Illinois	Inactive	2024-04-23	94867.33	alice.brown@example.com	-7713334475	Poor	0
18	EMP1017	Grace	Garcia	30	Finance	Texas	Pending	2024-11-28	NULL	grace.garcia@example.com	NULL	Average	1
19	EMP1018	Charlie	Garcia	NULL	Cloud Tech	Nevada	Inactive	2024-09-19	94726.46	charlie.garcia@example.com	-2044397358	Excellent	0
20	EMP1019	Charlie	Williams	35	Admin	Illinois	Pending	2020-01-02	91926.12	charlie.williams@example.com	-7107256365	Average	1
21	EMP1020	Bob	Miller	35	HR	New York	Active	2021-05-08	111460.67	bob.miller@example.com	-9411800536	Excellent	1
22	EMP1021	Bob	Jones	40	Admin	Nevada	Active	2020-09-19	96182.56	bob.jones@example.com	-5990048963	Good	0
23	EMP1022	Charlie	Johnson	35	Sales	Illinois	Inactive	2021-03-09	115023.94	charlie.johnson@example.com	-6528092427	Average	1
24	EMP1023	David	Garcia	35	Finance	Nevada	Pending	NULL	80667.01	NULL	-7572158623	Excellent	0
25	EMP1024	Bob	Brown	30	Admin	Texas	Inactive	2020-02-13	91181.61	bob.brown@example.com	-1502745643	Good	1
26	EMP1025	David	Miller	NULL	Cloud Tech	Nevada	Pending	2020-04-30	66025.92	david.miller@example.com	-9485625727	Average	0
27	EMP1026	Frank	Miller	25	Sales	New York	Inactive	2023-10-02	83979.01	frank.miller@example.com	-7032032073	Average	0
28	EMP1027	Heidi	Johnson	30	Finance	Florida	Active	2020-07-12	NULL	heidi.johnson@example.com	NULL	Excellent	1
29	EMP1028	Grace	Garcia	40	Finance	Nevada	Active	2023-10-11	95741.88	grace.garcia@example.com	-7712151341	Excellent	0
30	EMP1029	Alice	Brown	40	DevOps	California	Active	2021-05-24	63411.31	alice.brown@example.com	-2849939088	Excellent	1

4) Veri Kalitesi Analizi

Veri temizleme sürecinin etkinliğini ölçmek amacıyla çeşitli analizler yapılmıştır. Analizler data_result_reports klasörü içindeki bu scriptler ile gerçekleştirilmiştir: total_employee_counts.sql, problem_counts.sql, null_counts.sql

Bu analizler sayesinde “Kaç veri hatalıydı?”, “Hangi alanlarda sorun vardı?”, “Temizleme sonrası kaç değer NULL oldu?” gibi sorulara cevap üretilmeye çalışılmıştır.

	issue_type	issue_count
1	Invalid Age	300
2	Invalid Email	102
3	Invalid Salary	123
4	Invalid Join Date	102
5	Trim Needed In First_Name	102
6	Department Format Inconsistent	1020

	null_age	null_join_date	null_salary	null_email	null_phone
1	300	102	123	102	102

5) Sonuç ve Genel Değerlendirme

Bu projede, veri temizleme sürecinin veri analizi açısından ne kadar önemli olduğu uygulamalı olarak gösterilmiştir. Kirli verilerin doğrudan kullanılması hatalı sonuçlara yol açar, yapılan ETL işlemleri sayesinde veri daha güvenilir ve analiz edilebilir hale getirilmiştir.

Ayrıca bu projede; veri temizleme ve dönüşüm işlemlerinin dikkatli yapılması, orijinal verinin korunması gibi konulara da yer verilip önemli veri mühendisliği prensipleri uygulanmıştır.

PROJE 2 - Veritabanı Yedekleme ve Felaketten Kurtarma Planı

1) Giriş

Bu projede, veritabanı sistemlerinde veri güvenliğinin sağlanması amacıyla yedekleme ve felaketten kurtarma süreçleri incelenmiştir. Veri kaybı, sistem çökmesi veya kullanıcı hataları gibi durumlar veritabanı sistemleri için ciddi riskler oluşturur. Bu nedenle yedekleme ve etkili bir geri yükleme (restore) stratejisi büyük önem taşımaktadır.

Tüm süreçte MSSQL kullanılmıştır, araç olarak SQL Server Management Studio kullanılmıştır.

Projede, daha önce proje 5 için oluşturulan project_5_db adlı veritabanı kullanılmıştır. Veritabanı içerisinde çalışan(employee) verileri barındıran tablolar bulunmaktadır. Yedekleme işlemi bu veritabanı üzerinden yapılmıştır.

2) Yedekleme Türleri

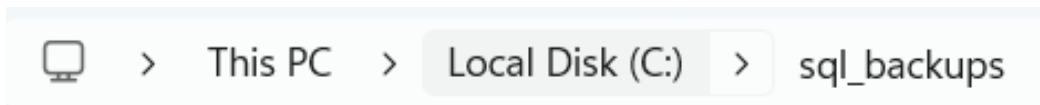
Veritabanı yedekleme işlemleri veri kaybını önlemek amacıyla sistemdeki verilerin kopyalanmasını ifade eder. Bu projede, aşağıda kısaca tanımları verilen yedekleme türlerinden tam yedekleme(full backup) ve fark yedekleme(differential backup) kullanılmıştır.

Tam Yedekleme (Full Backup): Tam yedekleme, veritabanının tamamının bir kopyasının alınmasıdır. Bu yöntem en güvenli yedekleme yöntemidir ve tek başına geri yükleme işlemi için yeterlidir.

Fark Yedekleme (Differential Backup): Fark yedekleme, son alınan tam yedekten sonra yapılan değişikliklerin yedeklenmesini sağlar. Bu sayede daha hızlı yedekleme işlemleri gerçekleştirilebilir.

Transaction Log Yedekleme: Transaction log yedekleme, veritabanında gerçekleşen işlemlerin kayıtlarını tutar ve veritabanının belirli bir zamana geri döndürülmesini sağlar.

Bütün backup'ların ve diğer bazı dosyaların saklanması için bir tane sql_backups klasörü oluşturulmuştur.



3) Genel Akış

3.1. Yedekleme İşlemleri

Sırasıyla 01_fullBackup.sql ve 02_diffBackup.sql scriptleri çalıştırılmıştır.

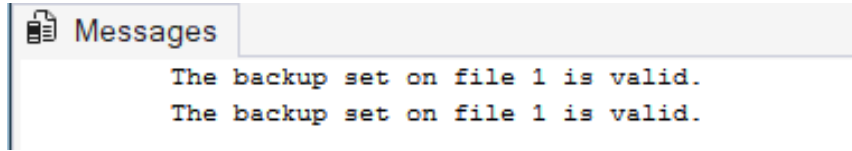
'C:\sql_backups\project_5_db_full.bak' şeklinde full backup,

'C:\sql_backups\project_5_db_diff.bak' şeklinde diff backup alınmıştır.

Full backup scriptinde aynı dosya adı varsa üstüne yeni yedek yazmak, backup medya bilgisini sıfırlayıp temizden yazmak, yüzde ilerlemeyi göstermek gibi ayarlar yapılmıştır.

3.2. Yedek Doğrulama ve Yedek İçeriğinin İncelenmesi

Alınan yedeğin doğruluğunu kontrol etmek için 03_verify.sql kullanılmıştır ve bunun ile yedek dosyasının geçerli olduğu doğrulanmıştır.



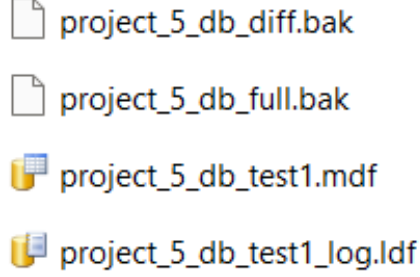
Yedek dosyasının içeriğini görmek için 04_filelist.sql kullanılmıştır ve bunun ile veritabanına ait veri ve log dosyalarının mantıksal isimleri elde edilmiştir. Bununla birlikte restore yaparken doğru isimlerin kullanılması sağlanır.

	LogicalName	PhysicalName	Type	FileGroupName	Size	MaxSize	FileId	CreateLSN	DropLSN	Uniqueld	ReadOnlyLSN	ReadWriteLSN	BackupSizeInBytes
1	project_5_db	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	D	PRIMARY	8388608	35184372080640	1	0	0	9C7A026C-2336-4AF7-AAD4-35AB86A7D8ED	0	0	5636096
2	project_5_db_log	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	L	NULL	8388608	2199023255552	2	0	0	6C13B30D-9C74-45EC-8C66-F2C6B7B93D1D	0	0	0

	LogicalName	PhysicalName	Type	FileGroupName	Size	MaxSize	FileId	CreateLSN	DropLSN	Uniqueld	ReadOnlyLSN	ReadWriteLSN	BackupSizeInBytes
1	project_5_db	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	D	PRIMARY	8388608	35184372080640	1	0	0	9C7A026C-2336-4AF7-AAD4-35AB86A7D8ED	0	0	589824
2	project_5_db_log	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	L	NULL	8388608	2199023255552	2	0	0	6C13B30D-9C74-45EC-8C66-F2C6B7B93D1D	0	0	0

3.3. Felaket Senaryosu Durumunda Veritabanının Geri Yüklenmesi

Bu aşamada, asıl veritabanında veri kaybı yaşandığı varsayılmıştır ve yedekten geri yükleme işlemi için 05_restore.sql kullanılmıştır. Bu işlem sonucunda veritabanı yeni bir isim ile geri yüklenmiştir. Bu isim project_5_db_test1'dir. Aşağıda görüldüğü üzere sql_backups klasörüne mdf (main data file) ve ldf (log data file) uzantılı dosyalarımız gelmiştir.



3.4. Kurtarma Sonrası Kontrol

Verilerin başarıyla geri yüklenip yüklenmediğini kontrol etmek için 06_test.sql kullanılmıştır, tabi ki bu script tüm satırları test etmese de düzenlenebilir ve istenilen yerler de test edilebilir. Kontrol SELECT ile yapılmıştır.

4) Sonuç ve Değerlendirmeler

Bu projede, veritabanı sistemlerinde veri güvenliğini sağlamak amacıyla yedekleme ve felaketten kurtarma süreçleri uygulamalı olarak incelenmiştir.

Gerçek dünyada veri kaybı; kullanıcı hataları, donanım arızaları veya yazılımsal problemler gibi birçok farklı sebepten kaynaklanabilmektedir. Bu nedenle, veritabanı sistemlerinde yedekleme yapılması ve gerektiği durumda veritabanının geri yüklenmesi kritik bir gerekliliktir.

Felaket senaryosu kapsamında, veritabanının zarar gördüğü varsayılmış ve alınan yedekler kullanarak veritabanı farklı bir isim altında yeniden oluşturulmuştur. Bu işlem sayesinde yedekleme sistemlerinin veri kaybı durumlarında nasıl etkili bir çözüm sunduğu açıkça görülmüştür.

PROJE 1 - Veritabanı Performans Optimizasyonu ve İzleme

1) Giriş

Bu projede, büyük hacimli veriler üzerinde çalışan veritabanı sisteminde performans analizleri gerçekleştirilmiş ve çeşitli optimizasyon yöntemleri uygulanmıştır. Veritabanı sistemlerinde performans, özellikle veri miktarının artmasıyla birlikte kritik bir konu haline gelmektedir. Yavaş sorgular kullanıcı deneyimini olumsuz etkileyebilmekte ve sistem kaynaklarının verimsiz kullanılmasına neden olabilmektedir.

Proje kapsamında yaklaşık 1 milyon satırlık işlem verisi içeren bir veri seti kullanılmıştır. SQL Server Management Studio kullanılmış, sorgu performansları analiz edilmiş, indeksleme yöntemleri uygulanmış, Dynamic Management Views (DMV) yardımıyla sistem davranışları incelenmiş ve çeşitli sorgular optimize edilmiştir.

Veri seti linki: <https://www.kaggle.com/datasets/mountboy/online-store-customer-transactions-1m-rows/data>

2) Veri Seti

Projede "Online Store Customer Transactions" adlı büyük bir veri seti kullanılmıştır. Veri seti MSSQL ortamına import edilerek project_1_db veritabanı altında online_store_transactions tablosu oluşturulmuştur.

Veri seti içerisinde Transaction_ID, Transaction_date, Gender, Age, Marital_status, State_names, Segment, Employees_status, Payment_method, Referral ve Amount_spent gibi sütunlar bulunmaktadır. Veri setinde toplamda yaklaşık 1 milyon satır bulunmaktadır. Bu set üzerinden indeksleme gibi işlemler için uygun bir test ortamı elde edilmiştir.

Aşağıda örnek olması amacıyla veri setinin ilk 10 satırı görsel olarak verilmiştir.

	Transaction_ID	Transaction_date	Gender	Age	Marital_status	State_names	Segment	Employees_status	Payment_method	Referral	Amount_spent
1	1000	2018-01-01 00:04:00.0000000	Female	39	Single	Oklahoma	Platinum	Unemployment	Card	0	1557.5
2	1001	2018-01-01 00:06:00.0000000	Male	34	Married	Hawaii	Basic	workers	PayPal	1	153.55
3	1002	2018-01-01 00:14:00.0000000	Female	53	Married	Iowa	Basic	self-employed	PayPal	1	2640.96
4	1003	2018-01-01 00:23:00.0000000	Male	33	Married	Wisconsin	Basic	self-employed	Card	1	293.58
5	1004	2018-01-01 00:25:00.0000000	Female	36	Married	Texas	Platinum	Employees	Card	0	1608.01
6	1005	2018-01-01 00:27:00.0000000	NULL	48	Married	North Carolina	Silver	self-employed	Other	NULL	1001.32
7	1006	2018-01-01 00:29:00.0000000	Female	55	Single	Ohio	Silver	Unemployment	PayPal	0	1553.61
8	1007	2018-01-01 00:29:00.0000000	Female	35	Married	Hawaii	Basic	workers	Other	1	1851.58
9	1008	2018-01-01 00:31:00.0000000	Male	18	Married	Nevada	Basic	Employees	Card	1	NULL
10	1009	2018-01-01 00:33:00.0000000	Female	50	Married	North Dakota	Basic	workers	Card	1	541.23

3) Performans Analizi ve Optimizasyon Süreci

3.1. Başlangıç Durumu

İlk aşamada herhangi bir özel indeks kullanılmadan sorgu çalıştırılmıştır. Ölçüm sırasında:

```
SET STATISTICS TIME ON;  
SET STATISTICS IO ON;
```

komutları kullanılmıştır.

Örneğin Texas eyaleti üzerinde gerçekleştirilen filtreleme ve toplulaştırma sorgusu analiz edilmiş, sorgunun CPU süresi, toplam çalışma süresi ve logical read değerleri kaydedilmiştir. Özellikle logical read değerleri sorgunun ne kadar veri okuduğunu göstermesi açısından önemli bir performans göstergesidir.

```
(1 row affected)  
Table 'online_store_transactions'. Scan count 1, logical reads 16597  
  
SQL Server Execution Times:  
CPU time = 188 ms, elapsed time = 203 ms.
```

3.2. Dynamic Management Views (DMV) ile Performans Takibi

Dynamic Management Views (DMV), SQL Server tarafından sağlanan ve veritabanının çalışma anındaki durumunu izlemeye yarayan sistem görünümleridir. DMV'ler sayesinde çalışan sorgular, indeks kullanımları, bellek tüketimi, işlem süreleri ve çeşitli performans istatistikleri gerçek zamanlı olarak görüntülenebilmektedir.

Bu projede DMV yapıları kullanılarak hem sorgu performansları hem de oluşturulan indekslerin kullanım durumları incelenmiştir. Özellikle **sys.dm_exec_query_stats** görünümü yardımıyla sorguların çalışma süreleri ve kaynak tüketimleri analiz edilmiş, **sys.dm_db_index_usage_stats** görünümü kullanılarak ise indekslerin seek ve scan davranışları gözlemlenmiştir.

Aşağıda sorgu tag'i ile birlikte yapılan sorgudan sonra **sys.dm_exec_query_stats** görünümünün execution_count, total_worker_time, total_elapsed_time için çalıştırılmasıyla çıkan sonucun görseli verilmiştir. Hemen altında ise daha hiç indeksleme işlemi yapılmadan **sys.dm_db_index_usage_stats** görünümünün kullanılmasıyla çıkan sonucun görseli verilmiştir.

	execution_count	total_worker_time	total_elapsed_time	text
1	1	170881	170889	/* PERF_TEST_TEXAS_SUM */ SELECT State_names, SU...
2	1	33623	33624	SELECT TOP 20 qs.execution_count, qs.total_worker_ti...

	database_name	table_name	index_name	user_seeks	user_scans	user_lookups	user_updates
1	project_1_db	online_store_transactions	PK_online_store_transactions	0	38	0	0

3.3. İndeks Yönetimi ve Sorgu İyileştirme

Performans iyileştirme amacıyla filtreleme yapılan alanlar üzerinde indeksleme çalışmaları gerçekleştirilmiştir. İlk olarak State_names alanı üzerinde indeks oluşturulmuştur. İndeksleme işlemleri sayesinde SQL Server'ın tam tablo taraması yapmak yerine doğrudan ilgili veri bloklarına erişmesi sağlanmıştır. İndeksleme işlemlerinden sonra aynı sorgular tekrar çalıştırılmış ve performans sonuçları karşılaştırılmıştır. Örneğin Texas eyaleti ile ilgili olan sorgu tekrar çalıştırıldığına state_names sütunu artık indeksli olduğu için performansta ciddi farklılıklar olmuştur. State_names indekslemesinden sonra transaction_date gibi alanlar da indekslenmiştir.

Yapılan ölçümlerde; logical read değerlerinde ciddi azalma, CPU kullanım sürelerinde düşüş, elapsed time değerlerinde iyileşme gözlemlenmiştir.

```
Table 'online_store_transactions'. Scan count 1, logical reads 41
```

```
SQL Server Execution Times:
```

```
CPU time = 0 ms, elapsed time = 7 ms.
```

Transaction_state indekslemesinden sonra yüksek olan user_scans azalmış, 0 olan user_seeks değeri 1'e çıkmıştır. Scan'de baştan sona tararken seek'de direkt ilgili olana gider. Bu yüzden DMV çıktılarında user seek değerinin artması ve user scan kullanımının azalması indeks optimizasyonunun başarılı olduğunu göstermektedir.

	database_name	table_name	index_name	user_seeks	user_scans	user_lookups	user_updates
1	project_1_db	online_store_transactions	PK_online_store_transactions	0	38	0	0
2	project_1_db	online_store_transactions	IX_transactions_state	1	0	0	0

4) Sonuç ve Genel Değerlendirme

Bu projede veritabanı performans analizi ve optimizasyon süreçleri uygulamalı olarak incelenmiştir. Dynamic Management Views aracı kullanılarak sorguların sistem üzerindeki etkileri analiz edilmiştir.

İndeksleme işlemlerinin sorgu performansı üzerindeki etkileri uygulamalı olarak gösterilmiş, filtreleme ve toplulaştırma işlemlerinde uygun indeks kullanımının önemli avantajlar sağlayabildiği görülmüştür. Bununla birlikte gereksiz tarama işlemlerinin azaltılması, logical read değerlerinin düşürülmesi ve CPU kullanımının optimize edilmesi sayesinde veritabanı sistemlerinde önemli performans kazanımları elde edilebileceği gösterilmiştir.

PROJE 3 - Veritabanı Güvenliği ve Erişim Kontrolü

1) Giriş

Veritabanı sistemleri, günümüzde kurumların ve uygulamaların en kritik bileşenlerinden biri haline gelmiştir. Bu sistemlerde saklanan veriler arasında hassas bilgiler de yer alabilmektedir. Bu nedenle verilerin saklanması yanında aynı zamanda yetkisiz erişimlere karşı korunması da büyük önem taşımaktadır.

Proje süresince MSSQL kullanılmıştır, araç olarak SQL Server Management Studio kullanılmıştır.

Projede, daha önce proje 5 için de kullanılan çalışan bilgilerini barındıran veri seti kullanılmıştır. Kısaca çalışmada rol tabanlı yetkilendirme mekanizmaları uygulanmış, hassas verilerin görüntülenmesi sınırlandırılmış, SQL Injection saldırılarına karşı korunma yöntemleri gösterilmiş ve kullanıcı aktivitelerinin izlenebilmesi amacıyla audit log mekanizması oluşturulmuştur. Böylece proje içerisinde veritabanı güvenliğinin temel bileşenleri uygulamalı olarak incelenmiştir.

2) Rol ve Erişim Yönetimi

Bu proje kapsamında rol tabanlı erişim kontrolü yaklaşımı kullanılmıştır. Farklı yetki seviyelerine sahip roller oluşturularak kullanıcıların veritabanı üzerindeki işlemleri sınırlandırılmıştır.

2.1. Rol Oluşturma

Öncelikle **readonly_user**, **readwrite_user** ve **admin_user** isimli üç farklı rol oluşturulmuştur. Böylece yalnızca veriyi görüntüleme yetkisine sahip kullanıcılar ile veri üzerinde değişiklik yapabilen kullanıcılar birbirinden ayrılmak istenmiştir. Admin (yönetici) rolü ise hassas veriler dahil bütün verileri görüntüleyebilen ve sistem üzerinde daha geniş yetkilere sahip olan rol olarak tanımlanmak istenmiştir.

	name	type_desc
1	readonly_user	DATABASE_ROLE
2	readwrite_user	DATABASE_ROLE
3	admin_user	DATABASE_ROLE

2.2. Kullanıcı Görünümleri ile Hassas Verilerin Korunması

Veritabanlarında yer alan tüm bilgilerin her kullanıcı tarafından görüntülenmesi güvenlik açısından uygun değildir. Özellikle maaş bilgileri, e-posta adresleri ve telefon numaraları gibi hassas veriler yalnızca yetkili kullanıcılar tarafından erişilebilir olmalıdır. Bu amaçla bir view yapısı kullanılarak belirli kolonların bazı kullanıcılardan gizlenmesi sağlanmak istenmiştir.

Bu noktada employee_public_view isimli bir görünüm oluşturulmuştur. Bu görünüm içerisinde çalışanlara ait genel bilgiler yer almakta ancak hassas veri olarak değerlendirilen maaş, e-posta ve telefon numarası alanları görüntülenmemektedir. Böylece belirli kullanıcıların yalnızca ihtiyaç duydukları verilere erişebilmesi sağlanmak istenir.

2.3. Rol Yetkilendirme

Projede GRANT, DENY komutları kullanılarak kullanıcı ve rollerin erişim hakları ayrıntılı biçimde ayarlanmıştır.

Read_only kullanıcı bütün veriyi görüntüleyemez sadece public görüntüyü okuyabilir. **Read_write** kullanıcı için de veri görüntüleme konusunda benzer bir durum geçerlidir ama ek olarak veri ekleme ya da veriyi güncelleme özelliğine de sahiptir, yani kısacası yazma işlemi yapabilir. **Admin** kullanıcıda durum biraz daha farklıdır, sadece public veriyi değil içinde hassas verilerin de bulunduğu tüm veriyi görüntüleyebilir. Ayrıca veri ekleme ve güncellemenin yanında veriyi silme yetkisine de sahiptir.

	role_name	permission_name	state_desc	object_name
1	admin_user	DELETE	GRANT	employee_data
2	admin_user	INSERT	GRANT	employee_data
3	admin_user	SELECT	GRANT	employee_data
4	admin_user	UPDATE	GRANT	employee_data
5	admin_user	SELECT	GRANT	employee_public_view
6	readonly_user	SELECT	DENY	employee_data
7	readonly_user	SELECT	GRANT	employee_public_view
8	readwrite_user	INSERT	GRANT	employee_data
9	readwrite_user	SELECT	DENY	employee_data
10	readwrite_user	UPDATE	GRANT	employee_data
11	readwrite_user	SELECT	GRANT	employee_public_view

3) SQL Injection

SQL Injection, kullanıcı tarafından girilen verilerin doğrudan SQL sorgularının içerisine eklenmesi sonucunda ortaya çıkan yaygın bir güvenlik açığıdır. Bu tür saldırılar sayesinde saldırganlar veritabanındaki hassas bilgilere erişebilir, yetkisiz işlemler gerçekleştirebilir veya sistem üzerinde çeşitli değişiklikler yapabilirler.

SQL Injection saldırısına açık bir sorgu örneği oluşturulmuş ve sonrasında parametrelili sorgular kullanılarak aynı işlemin güvenli bir şekilde nasıl gerçekleştirilebileceği gösterilmiştir.

3.1. Güvensiz Dinamik SQL Sorguları

Bu saldırıya örnek olarak kullanıcı girdisinin doğrudan sorgu içerisine eklendiği bir dinamik SQL örneği oluşturulmuştur. Örnekte kullanılan yöntem SQL Injection saldırılarına açık olup, kötü niyetli kullanıcıların sorgunun çalışma mantığını değiştirebilmesine olanak sağlayabilmektedir. Bu nedenle kullanıcı girdilerinin doğrudan SQL ifadelerine eklenmesi güvenli bir yaklaşım olarak kabul edilmemektedir.

```
1 DECLARE @name NVARCHAR(100);
2 DECLARE @sql NVARCHAR(MAX);
3
4 -- SET @name = 'Bob';
5 SET @name = '' OR 1=1 --';
6
7
8 SET @sql =
9 'SELECT *
10 FROM employee_data
11 WHERE First_Name = '' + @name + ''';
12
13 PRINT @sql;
14
15 EXEC(@sql);
```

Örnekte görüldüğü üzere `'' OR 1=1 --'` ifadesi ilgili değişkenin değeri olmaktan ziyade sorguyu bypass edecek bir ifade olarak kullanılmıştır ve sonuç olarak WHERE ifadesindeki koşulun her halükarda sağlanması sebebiyle tablonun hepsine SELECT işlemi yapılabilmektedir. Yani bu saldırı yönteminin kullanılması durumunda projede kullanılan bütün WHERE komutları bypass edilme ihtimaliyle karşı karşıyadır. Örnekte bu değişken name ifadesi olsa da bu saldırı her değişken üzerinden yapılabilir, şifreler koşulları bypass edilebilir.

3.2. Parametrelili Sorgular ile Saldırıdan Korunma

Parametrelili sorgularda kullanıcı girdileri SQL kodunun bir parçası olarak değil, veri olarak değerlendirilmektedir. Böylece kullanıcı tarafından gönderilen özel karakterler veya SQL ifadeleri sorgunun çalışma mantığını değiştirememektedir. Bu proje kapsamında SQL Server'ın sp_executesql yapısı kullanılarak parametrelili sorgu örneği geliştirilmiştir. Böylece kullanıcı girdileri güvenli bir şekilde işlenmiş ve SQL Injection saldırılarının önüne geçilmiştir.

Kısacası kod ve veri kısımlarını birbirinden ayıran bu yöntemin kullanılması ile istenilen sorgunun bypass edilerek çalıştırılmasına izin verilmemiştir.

```
1 DECLARE @name NVARCHAR(100);
2
3 --SET @name = 'Bob';
4 SET @name = '' OR 1=1 --';
5
6 EXEC sp_executesql
7 N'
8 SELECT *
9 FROM employee_data
10 WHERE First_Name = @name
11 ',
12 N'@name NVARCHAR(100)',
13 @name;
```


4) Audit Logları ve Kullanıcı İşlemlerinin Takibi

Veritabanı güvenliğinin önemli bileşenlerinden biri de kullanıcı aktivitelerinin izlenebilmesidir. Kullanıcıların gerçekleştirdiği işlemlerin kayıt altına alınması sayesinde sistem üzerinde meydana gelen değişiklikler takip edilebilmektedir ve bu kayıtlar aynı zamanda güvenlik incelemeleri açısından da önemli bilgiler sağlamaktadır.

Bu proje kapsamında kullanıcı işlemlerinin takip edilebilmesi amacıyla bir audit log mekanizması oluşturulmuştur. İlk olarak gerçekleştirilen işlemlerin saklanacağı bir audit tablosu hazırlanmış, daha sonra çalışan verilerinin güncellenmesi durumunda otomatik kayıt oluşturacak bir trigger geliştirilmiştir. Böylece işlemler otomatik olarak kayıt altına alınabilmektedir.

Oluşturulan audit mekanizması sayesinde gerçekleştirilen işlemin türü, ilgili çalışan kaydı, işlemi gerçekleştiren kullanıcı bilgisi ve işlem zamanı sistem tarafından saklanabilmektedir. Bu yaklaşım sayesinde veritabanında meydana gelen değişikliklerin geçmişi takip edilebilmekte ve gerektiğinde geriye dönük incelemeler yapılabilmektedir.

	audit_id	operation_type	employee_id	executed_by	execution_time
1	1	UPDATE	EMP1000	DESKTOP-76G83CC\EMRE	2026-06-03 13:30:08.560
2	2	UPDATE	EMP1000	DESKTOP-76G83CC\EMRE	2026-06-05 16:49:29.647
3	3	UPDATE	EMP1015	DESKTOP-76G83CC\EMRE	2026-06-05 16:49:29.650
4	4	UPDATE	EMP1100	DESKTOP-76G83CC\EMRE	2026-06-05 16:52:43.160

Test etmek amacıyla veritabanında yapılan birkaç güncellemenin sonucunda işlemlerin audit log tablosuna başarıyla kaydedildiği görülmüştür. Bu sayede kullanıcı aktivitelerinin izlenebilmesi ve gerçekleştirilen işlemlerin kayıt altına alınabilmesi sağlanmıştır.

5) Sonuç ve Genel Değerlendirme

Bu proje kapsamında rol tabanlı yetkilendirme yapısı oluşturulmuş, hassas verilerin korunması amacıyla View yapıları kullanılmış ve farklı kullanıcı grupları için farklı erişim seviyeleri tanımlanmıştır. Ayrıca SQL Injection saldırılarının çalışma mantığı incelenmiş ve parametrelili sorgular kullanılarak bu tür saldırılara karşı alınabilecek önlemler gösterilmiştir. Bunun yanında audit log mekanizması oluşturularak kullanıcı işlemlerinin kayıt altına alınması sağlanmıştır.

Yapılan çalışmalar sonucunda veritabanı güvenliğinin yetkilendirme, veri gizliliği, saldırılara karşı korunma ve işlem takibi gibi birçok bileşenden oluştuğu görülmüştür. Projede uygulanan yöntemler ile daha güvenli ve yönetilebilir bir veritabanı ortamı oluşturulmuştur.

PROJE 6 - Veritabanı Yükseltme ve Sürüm Yönetimi

1) Giriş

Bu proje kapsamında SQL Server Management Studio kullanılarak veritabanı sürüm yönetimi süreçleri incelenmiştir. Önceki projelerin bazılarında kullanılan employee(çalışan) verilerinin bulunduğu set kullanılmıştır. Başlangıçta oluşturulan veritabanı yapısı farklı sürümlere yükseltilmiş, gerçekleştirilen şema değişiklikleri kayıt altına alınmış ve gerektiğinde önceki sürümlere geri dönüş yapılabilmesi için rollback(geri dönüş) planları hazırlanmıştır. Böylece veritabanı geliştirme süreçlerinde kullanılan temel sürüm yönetimi yaklaşımları uygulamalı olarak gösterilmiştir.

2) Veritabanı Sürüm Yönetimi ve Sürüm Geçiş Süreci

Veritabanı sürüm yönetimi, veritabanı şemasında zaman içerisinde gerçekleştirilen değişikliklerin planlı ve izlenebilir bir şekilde yönetilmesini ifade etmektedir. Sürüm yönetimi sayesinde veritabanı yapısındaki değişiklikler kayıt altına alınabilir ve farklı sürümler arasında geçiş yapılabilir. Bu noktada temel amaç, mevcut yapıyı kontrollü bir şekilde geliştirerek yeni ihtiyaçlara uygun hale getirmek olmuştur. Bu süreçte yapılan değişikliklerin planlı bir şekilde uygulanması ve kayıt altına alınması önem taşımıştır.

Proje kapsamında veritabanı yapısı üç farklı sürüm üzerinden ele alınmış ve her sürümde gerçekleştirilen değişiklikler ayrı ayrı incelenmiştir. İlk sürüm başlangıç yapısını temsil ederken, sonraki sürümlerde mevcut yapı üzerinde çeşitli düzenlemeler gerçekleştirilmiştir. Böylece veritabanının zaman içerisindeki gelişimi ve sürümler arasındaki farklar uygulamalı olarak gösterilmiştir.

2.1. Sürüm v1

İlk sürüm, projede kullanılan başlangıç veritabanı yapısını temsil etmektedir. Bu sürümde çalışan bilgileri tek bir tablo içerisinde saklanmakta ve departman ile bölge bilgileri Department_Region isimli tek bir kolon içerisinde tutulmaktadır. Sistemin ilk hali herhangi bir şema değişikliği içermemekte ve sonraki sürümlerde yapılacak geliştirmeler için temel oluşturmaktadır. Sürüm v1, veritabanının başlangıç noktası olarak değerlendirilmiştir ve sonraki sürümlerde gerçekleştirilen tüm değişiklikler bu temel yapı üzerinden uygulanmıştır.

	Employee_ID	First_Name	Last_Name	Age	Status	Join_Date	Salary	Email	Phone	Performance_Score	Remote_Work	Department_Region
1	EMP1000	Bob	Davis	25	Active	4/2/2021	59767.65	bob.davis@example.com	-1651623197	Average	TRUE	DevOps-California
2	EMP1001	Bob	Brown	NULL	Active	7/10/2020	65304.66	bob.brown@example.com	-1898471390	Excellent	TRUE	Finance-Texas
3	EMP1002	Alice	Jones	NULL	Pending	12/7/2023	88145.9	alice.jones@example.com	-5596363211	Good	TRUE	Admin-Nevada
4	EMP1003	Eva	Davis	25	Inactive	11/27/2021	69450.99	eva.davis@example.com	-3476490784	Good	TRUE	Admin-Nevada
5	EMP1004	Frank	Williams	25	Active	1/5/2022	109324.61	frank.williams@example.com	-1586734256	Poor	FALSE	Cloud Tech-Florida
6	EMP1005	Alice	Garcia	40	Inactive	6/10/2020	88642.84	alice.garcia@example.com	-5409003485	Good	FALSE	Sales-Texas
7	EMP1006	Frank	Jones	NULL	Active	4/3/2020	96288.43	frank.jones@example.com	-4518376063	Good	FALSE	Admin-Nevada
8	EMP1007	Bob	Jones	30	Inactive	7/17/2022	94497.91	bob.jones@example.com	-4134327559	Average	TRUE	Cloud Tech-Florida
9	EMP1008	Frank	Davis	35	Inactive	12/8/2023	115565.82	frank.davis@example.com	-4177656123	Excellent	TRUE	Admin-Nevada
10	EMP1009	Charlie	Johnson	NULL	Active	8/4/2022	76561.88	charlie.johnson@example.com	-8156985699	Excellent	TRUE	DevOps-New York

2.2. Sürüm v2

İkinci sürümde veri modelinin daha düzenli ve yönetilebilir hale getirilmesi amacıyla Department ve Region isimli yeni kolonlar eklenmiştir. Daha önce tek bir kolon içerisinde tutulan departman ve bölge bilgileri ayrıştırılarak ayrı alanlarda saklanmaya başlanmıştır. Bu sayede verilerin sorgulanması ve yönetilmesi daha kolay hale gelmiştir. Yeni kolonların eklenmesinin ardından mevcut Department_Region verileri parçalanarak ilgili alanlara aktarılmıştır. Böylece hem eski yapı korunmuş hem de yeni veri modeli kullanıma hazır hale getirilmiştir. v3 sürümüne geçilmeden önce Department_Region sütunu silinmemiştir.

	Employee_ID	First_Name	Last_Name	Age	Status	Join_Date	Salary	Email	Phone	Performance_Score	Remote_Work	Department_Region	Department	Region
1	EMP1000	Bob	Davis	25	Active	4/2/2021	59767.65	bob.davis@example.com	-1651623197	Average	TRUE	DevOps-California	DevOps	California
2	EMP1001	Bob	Brown	NULL	Active	7/10/2020	65304.66	bob.brown@example.com	-1898471390	Excellent	TRUE	Finance-Texas	Finance	Texas
3	EMP1002	Alice	Jones	NULL	Pending	12/7/2023	88145.9	alice.jones@example.com	-5596363211	Good	TRUE	Admin-Nevada	Admin	Nevada
4	EMP1003	Eva	Davis	25	Inactive	11/27/2021	69450.99	eva.davis@example.com	-3476490784	Good	TRUE	Admin-Nevada	Admin	Nevada
5	EMP1004	Frank	Williams	25	Active	1/5/2022	109324.61	frank.williams@example.com	-1586734256	Poor	FALSE	Cloud Tech-Florida	Cloud Tech	Florida
6	EMP1005	Alice	Garcia	40	Inactive	6/10/2020	88642.84	alice.garcia@example.com	-5409003485	Good	FALSE	Sales-Texas	Sales	Texas
7	EMP1006	Frank	Jones	NULL	Active	4/3/2020	96288.43	frank.jones@example.com	-4518376063	Good	FALSE	Admin-Nevada	Admin	Nevada
8	EMP1007	Bob	Jones	30	Inactive	7/17/2022	94497.91	bob.jones@example.com	-4134327559	Average	TRUE	Cloud Tech-Florida	Cloud Tech	Florida
9	EMP1008	Frank	Davis	35	Inactive	12/8/2023	115565.82	frank.davis@example.com	-4177656123	Excellent	TRUE	Admin-Nevada	Admin	Nevada
10	EMP1009	Charlie	Johnson	NULL	Active	8/4/2022	76561.88	charlie.johnson@example.com	-8156985699	Excellent	TRUE	DevOps-New York	DevOps	New York

2.3. Sürüm v3

Üçüncü sürümde geçiş süreci tamamlanmış ve artık ihtiyaç duyulmayan Department_Region kolonu sistemden kaldırılmıştır. Departman ve bölge bilgileri ayrı alanlarda tutulduğu için eski kolon gereksiz hale gelmiştir. Böylece veritabanı yapısı sadeleştirilmiş ve veri tekrarının önüne geçilmiştir. Aynı sürümde çalışanların görev bilgilerinin tutulabilmesi amacıyla Job_Title isimli yeni bir kolon eklenmiştir. Bu değişiklik ile veritabanı yapısı yeni iş ihtiyaçlarına uygun hale getirilmiştir. Bütün bunlar dolayısıyla veritabanının yeni sürümüne yükseltme süreci tamamlanmıştır.

	Employee_ID	First_Name	Last_Name	Age	Status	Join_Date	Salary	Email	Phone	Performance_Score	Remote_Work	Department	Region	Job_Title
1	EMP1000	Bob	Davis	25	Active	4/2/2021	59767.65	bob.davis@example.com	-1651623197	Average	TRUE	DevOps	California	NULL
2	EMP1001	Bob	Brown	NULL	Active	7/10/2020	65304.66	bob.brown@example.com	-1898471390	Excellent	TRUE	Finance	Texas	NULL
3	EMP1002	Alice	Jones	NULL	Pending	12/7/2023	88145.9	alice.jones@example.com	-5596363211	Good	TRUE	Admin	Nevada	NULL
4	EMP1003	Eva	Davis	25	Inactive	11/27/2021	69450.99	eva.davis@example.com	-3476490784	Good	TRUE	Admin	Nevada	NULL
5	EMP1004	Frank	Williams	25	Active	1/5/2022	109324.61	frank.williams@example.com	-1586734256	Poor	FALSE	Cloud Tech	Florida	NULL
6	EMP1005	Alice	Garcia	40	Inactive	6/10/2020	88642.84	alice.garcia@example.com	-5409003485	Good	FALSE	Sales	Texas	Project Manager
7	EMP1006	Frank	Jones	NULL	Active	4/3/2020	96288.43	frank.jones@example.com	-4518376063	Good	FALSE	Admin	Nevada	NULL
8	EMP1007	Bob	Jones	30	Inactive	7/17/2022	94497.91	bob.jones@example.com	-4134327559	Average	TRUE	Cloud Tech	Florida	NULL
9	EMP1008	Frank	Davis	35	Inactive	12/8/2023	115565.82	frank.davis@example.com	-4177656123	Excellent	TRUE	Admin	Nevada	NULL
10	EMP1009	Charlie	Johnson	NULL	Active	8/4/2022	76561.88	charlie.johnson@example.com	-8156985699	Excellent	TRUE	DevOps	New York	NULL

3) Geri Dönüş (Rollback) Planı

Sürüm yönetimi süreçlerinde yalnızca yükseltme adımlarının değil, geri dönüş planlarının da önceden hazırlanması önemlidir.

Bu proje kapsamında her sürüm geçişi için olası geri dönüş senaryoları değerlendirilmiştir. Sürüm v3'den sürüm v2'ye dönüş yapılması gerektiğinde Job_Title alanının kaldırılması ve Department_Region alanının yeniden oluşturulması bir rollback script (**rollback_v3_to_v2.sql**)

aracılığı ile planlanmıştır. Benzer şekilde sürüm v2'den sürüm v1'e dönüş için eklenen kolonların kaldırılmasına yönelik rollback scripti (**rollback_v2_to_v1.sql**) hazırlanmıştır. Böylece veritabanı yapısı kontrollü bir şekilde önceki sürümlere döndürülebilecek duruma gelmiştir.

Hazırlanan rollback planları sayesinde sürüm yükseltme işlemlerinin daha güvenli bir şekilde yönetilmesi sağlanmıştır. Olası bir hata durumunda sistemin önceki kararlı sürüme döndürülebilmesi, sürüm yönetimi sürecinin önemli bir parçasını oluşturmuştur.

4) DDL Trigger ile Şema Değişikliklerinin Takibi

Bu proje kapsamında veritabanı üzerinde gerçekleştirilen versiyonlama işlemlerinin takip edilebilmesi amacıyla bir DDL (Data Definition Language) Trigger geliştirilmiştir. Oluşturulan trigger sayesinde şema değişikliklerine ait bilgiler otomatik olarak kayıt altına alınmış ve ayrı bir log tablosunda saklanmıştır. Böylece sürüm yükseltme veya geri alma süreçlerinde yapılan işlemlerin geçmişi izlenebilir hale getirilmiştir.

	log_id	event_type	object_name	object_type	sql_command	changed_by	changed_at
1	8	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data DROP COLUMN ...	DESKTOP-76G83CCIE	2026-06-06 15:36:40.977
2	7	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data DROP COLUMN ...	DESKTOP-76G83CCIE	2026-06-06 15:36:40.970
3	6	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data DROP COLUMN ...	DESKTOP-76G83CCIE	2026-06-06 15:36:20.153
4	5	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data ADD Department...	DESKTOP-76G83CCIE	2026-06-06 15:36:20.123
5	3	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data DROP COLUMN ...	DESKTOP-76G83CCIE	2026-06-06 15:36:01.427
6	4	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data ADD Job_Title N...	DESKTOP-76G83CCIE	2026-06-06 15:36:01.427
7	2	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data ADD Region NVA...	DESKTOP-76G83CCIE	2026-06-06 15:35:42.013
8	1	ALTER_TABLE	employee_versioning_data	TABLE	ALTER TABLE employee_versioning_data ADD Department...	DESKTOP-76G83CCIE	2026-06-06 15:35:42.010

Yapılan versiyon yükseltmeler ve geri almalar sonucunda log tablosunda versiyonlama değişikliklerinin başarıyla kayıt altına alındığı görülmüştür. Bu sayede sürüm yönetimi sürecinde gerçekleştirilen işlemler daha kolay takip edilebilmiş ve veritabanı değişikliklerinin denetlenebilirliği artırılmıştır.

5) Sonuç ve Genel Değerlendirmeler

Projenin işleyişinde başlangıç sürümü temel alınarak yeni sürümler oluşturulmuş, veritabanı şemasında gerçekleştirilen değişiklikler kontrollü bir şekilde uygulanmıştır. Sürüm yükseltme işlemleri sırasında yeni alanlar eklenmiş, veri yapısı iyileştirilmiş ve ihtiyaç duyulmayan alanlar sistemden kaldırılmıştır. Ayrıca sürüm geçişlerinin güvenli bir şekilde yönetilebilmesi amacıyla geri dönüş planları hazırlanmış ve DDL Trigger mekanizması kullanılarak şema değişiklikleri kayıt altına alınmıştır.

Yapılan çalışmalar sonucunda veritabanı sürüm yönetiminin yalnızca şema değişikliklerini uygulamaktan ibaret olmadığı, aynı zamanda değişikliklerin izlenmesi ve gerektiğinde geri alınabilmesi süreçlerini de kapsadığı görülmüştür.